

OpenMC-Informed Reactor Model

Physics, Numerics, and Web Application Coupling

Contents

1	Purpose and model boundary	2
2	From the first diffusion model to OpenMC	2
3	Current OpenMC reactor and published reference data	2
4	Overview page: point-kinetics transient model	3
4.1	Governing equations	3
4.2	Constants used by the web app	4
4.3	Initial conditions	4
4.4	How rod motion enters the point model	5
4.5	Thermal reactivity feedback	5
4.6	Numerical method	5
4.7	How the Overview transient is operationalized	7
5	Resolved multigroup diffusion for the Core page	7
5.1	Overview of the pipeline	7
5.2	MGXS export from OpenMC	8
5.3	Adapter: from OpenMC cells to diffusion regions	9
5.4	Finite-volume multigroup diffusion solver	10
5.4.1	Spatial discretization	10
5.4.2	Production-mesh qualification	11
5.4.3	Boundary conditions	11
5.4.4	Eigenvalue iteration	11
5.5	CE-referenced superhomogenization	12
5.5.1	Post-solve power-shape correction	13
5.5.2	Rod-worth scan	14
5.6	Persistent preparation cache	14
5.7	Integration with the web application	15
5.8	Modelica thermal-hydraulics coupling	15
5.9	Current status and verification	16
6	How the modeling layers relate	17
7	Important assumptions and limitations	17
8	Conclusion	18

1 Purpose and model boundary

This note is the physics and numerical-method reference for the *current* Reactor Kinetics Lab web application. It follows the active modeling path from the continuous-energy OpenMC reactor through published reference data, point kinetics, the resolved four-group diffusion solver, and the Modelica thermal-hydraulics coupling.

The notebooks are analysis and reference-data workspaces. The Python backend is the runtime source of truth and the React frontend presents its state. The retired homogeneous one-group model and its old Core and Transient Diffusion backends are documented separately in `legacy_homogeneous_reactor.pdf`; they are not frontend models.

2 From the first diffusion model to OpenMC

The project began with a homogeneous buckling estimate followed by a one-group, two-dimensional annular diffusion model. That work was useful for learning leakage, critical-size searches, rod-worth scans, and sparse eigenvalue methods. It also exposed the weakness of selecting engineering-effective one-group constants by hand: spectral effects, resolved fuel geometry, and control-rod behavior could not be defended independently.

The next modeling stage therefore used continuous-energy Monte Carlo in OpenMC. An FRM-II-inspired involute natural-uranium concept was explored first. A concentric annular arrangement was then selected because it preserves the main fuel/moderator heterogeneity while remaining practical for resolved cell-wise MGXS generation and an axisymmetric diffusion model. The design is inspired by research-reactor layouts; it is not a replica of a licensed facility.

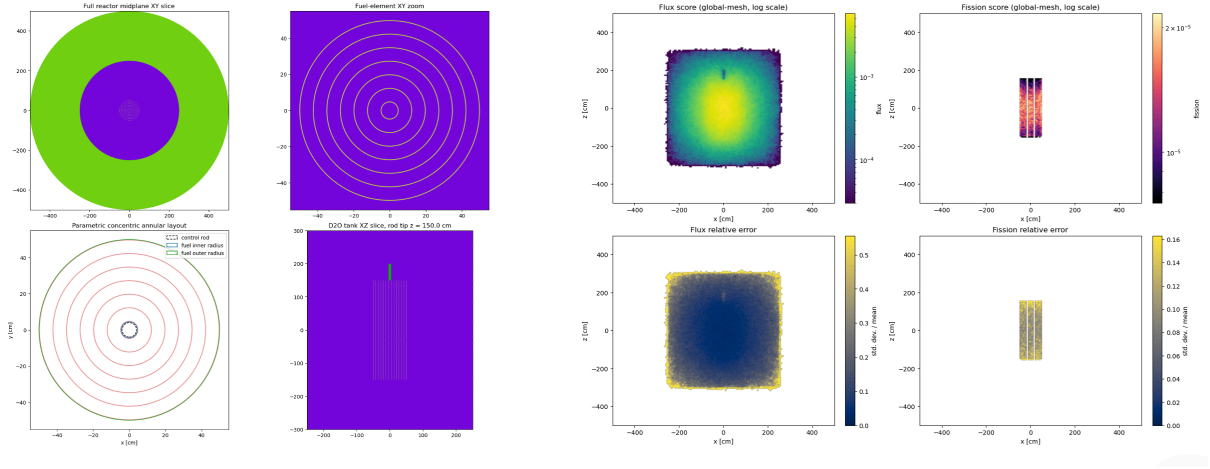


Figure 1: Development from exploratory reactor geometry to the resolved concentric OpenMC model.

3 Current OpenMC reactor and published reference data

The runtime neutronics reference is the tracked four-group publication under `openmc/reference_data/concentric/group_sweep/group_4`. Its `model.xml` preserves seven fuel-ring cells, moderator and coolant regions, a parked control-rod cell, and the surrounding D₂O and H₂O regions. The active fuel height is 300 cm and the fuel-element radius is 50 cm; the published diffusion geometry

extends to a 250 cm moderator radius and a 500 cm reflector radius. These dimensions come from the publication, not from the legacy one-group calibration.

OpenMC is run offline. Normal web requests consume compact, versioned artifacts rather than launching a Monte Carlo calculation:

Artifact	Runtime loader	Use
<code>mgxs_constants.json</code>	<code>kinetics_reference.py</code> and the MGXS adapter	Delayed-neutron constants, generation time, resolved multi-group cross sections, CE flux tallies, and the clean CE power mesh
<code>model.xml</code>	<code>openmc_mgxs_adapter.py</code>	Resolved cell names and radial/axial material boundaries
<code>rod_worth.csv</code>	<code>rod_worth.py</code>	Continuous-energy rod worth used by point kinetics
<code>reactivity_coefficients.csv</code>	the reactivity-coefficient loader	Fuel-temperature, moderator-temperature, and moderator-density feedback slopes

Two clean-core references must not be conflated. The independent continuous-energy rod scan starts at $k_{\text{eff}} = 1.00224587$, or +224.084 pcm, and defines the point-kinetics rod-worth curve. The continuous-energy run associated with the published four-group MGXS has $k_{\text{eff}} = 1.00142016 \pm 21.75$ pcm and is the comparison reference for the Core-page diffusion calculation.

4 Overview page: point-kinetics transient model

4.1 Governing equations

The Overview page uses the standard six-group delayed-neutron point-kinetics system:

$$\frac{dn}{dt} = \frac{\rho - \beta}{\Lambda} n + \sum_{i=1}^6 \lambda_i C_i, \quad (1)$$

$$\frac{dC_i}{dt} = \frac{\beta_i}{\Lambda} n - \lambda_i C_i, \quad i = 1, \dots, 6. \quad (2)$$

Here:

- $n(t)$ is the normalized neutron population,
- ρ is reactivity in $\Delta k/k$,
- $\beta = \sum_i \beta_i$ is the effective delayed-neutron fraction,
- Λ is the prompt neutron generation time,
- C_i is precursor concentration for delayed group i ,
- λ_i is the delayed-group decay constant.

4.2 Constants used by the web app

The current production constants in `config.py` are loaded from the published four-group OpenMC MGXS export `openmc/reference_data/concentric/group_sweep/group_4/outputs/mgxs_constants.json`. The backend averages the fuel-ring delayed-neutron data with a fission-source weight,

$$w_j = \sum_g (\nu \Sigma_f)_{j,g} \Phi_{j,g},$$

where j indexes the resolved fuel-ring domains. The fuel-ring β_i values are *averaged*, not summed across rings. The prompt neutron generation time is taken from the export's `prompt_generation_time_s` tally.

The resulting constants are:

$$\begin{aligned}\beta_{\text{eff}} &= 0.00678910882978 = 678.91 \text{ pcm}, \\ \Lambda &= 0.00417212833712 \text{ s}.\end{aligned}$$

The six delayed groups are:

Group	β_i	$\lambda_i \text{ [s}^{-1}\text{]}$
1	0.000227917094163	0.0133443203744
2	0.00119534670498	0.0326776283533
3	0.00115198689522	0.120914791504
4	0.00262514724228	0.304202879275
5	0.00112075519129	0.855415421400
6	0.000467955701846	2.87295916880

Displayed output is scaled from the nominal operating point

$$P_0 = 20 \text{ MW}_{\text{th}}, \quad \Phi_0 = 1.5 \times 10^{12} \text{ n cm}^{-2} \text{ s}^{-1},$$

using

$$P(t) = P_0 n(t), \quad \Phi(t) = \Phi_0 n(t).$$

The 20 MW value is the design point shared with the Modelica model. The absolute flux scale is different: it is still the peak-flux normalization inherited from the retired one-group Estimate 2 calculation. It is not derived from the current OpenMC model and should be replaced by an OpenMC-consistent absolute normalization in future work. This legacy scale changes the displayed value of total flux, but not the normalized point- kinetics solution.

4.3 Initial conditions

On construction and reset, the backend initializes

$$n(0) = 1, \quad C_i(0) = \frac{\beta_i}{\Lambda \lambda_i} n(0).$$

The rod starts at the interpolated continuous-energy critical insertion, $x_{\text{crit}} = 26.5323\%$, time starts at zero, and the displayed power is therefore 20 MW. The thermal adapter is reset at that power. If a compatible FMU starts successfully, its initial effective fuel temperature, moderator temperature, and moderator density are captured as the zero-feedback reference. This makes the coupled reset state internally critical even when the FMU initial state differs from the temperatures used to generate the OpenMC coefficient slopes.

4.4 How rod motion enters the point model

The point-kinetics solver does not compute rod worth from first principles during runtime. Instead it loads the OpenMC continuous-energy rod-worth table `openmc/reference_data/concentric/rod_scan/results/rod_worth.csv`. Reactivity is computed as

$$\rho_{\text{tot,pcm}}(x) = \rho_{\text{clean,pcm}} + \Delta\rho_{\text{rod,pcm}}(x) + \rho_{\text{scram,pcm}} + \rho_{\text{fb,pcm}},$$

where $x \in [0, 1]$ is rod insertion fraction. In the current calibration:

$$\begin{aligned} k_{\text{clean}} &= 1.00224587, \\ \rho_{\text{clean}} &= +224.084 \text{ pcm}, \\ x_{\text{crit}} &= 0.265323, \\ \Delta\rho_{\text{rod}}(1.0) &= -1350.73 \text{ pcm}, \\ \rho_{\text{scram}} &= -450 \text{ pcm when latched.} \end{aligned}$$

The backend interpolates linearly between the insertion points stored in the CSV. The older diffusion-derived rod-worth curve is not used here. Legacy calibration data remain only in the retired one-group backends and in the nominal flux and geometry metadata described above.

4.5 Thermal reactivity feedback

The point model also adds thermal feedback when the Modelica FMU reports all three effective feedback signals: `T_fuelEff`, `T_moderatorEff`, and `rho_m_eff`. The coefficients are loaded from `openmc/reference_data/concentric/reactivity_coefficients/results/reactivity_coefficients.csv`:

$$\begin{aligned} \alpha_f &= -2.138272335 \text{ pcm/K}, \\ \alpha_{m,T} &= -10.72767894 \text{ pcm/K}, \\ \alpha_{m,\rho} &= 20993.04779 \text{ pcm/(g/cm}^3\text{)}. \end{aligned}$$

At reset, the current FMU effective state is stored as the zero-feedback reference. Runtime feedback is then

$$\rho_{\text{fb,pcm}} = \alpha_f (T_{f,\text{eff}} - T_{f,0}) + \alpha_{m,T} (T_{m,\text{eff}} - T_{m,0}) + \alpha_{m,\rho} (\rho_{m,\text{eff}} - \rho_{m,0}).$$

This preserves the existing critical startup behavior while still using the OpenMC finite-difference slopes. Thermal feedback is **FMI-only**: if the FMU is unavailable and the fallback thermal model is active, the feedback terms are reported as zero and are not added to reactivity.

4.6 Numerical method

The point-kinetics equations are stiff, so the backend does not use explicit Euler. Instead `engine.py` advances both the neutron population and the precursors implicitly at the new time level and then algebraically eliminates the precursor unknowns.

Start from the backward-Euler discretization

$$\frac{n^{k+1} - n^k}{\Delta t} = \frac{\rho - \beta}{\Lambda} n^{k+1} + \sum_{i=1}^6 \lambda_i C_i^{k+1}, \quad (3)$$

$$\frac{C_i^{k+1} - C_i^k}{\Delta t} = \frac{\beta_i}{\Lambda} n^{k+1} - \lambda_i C_i^{k+1}. \quad (4)$$

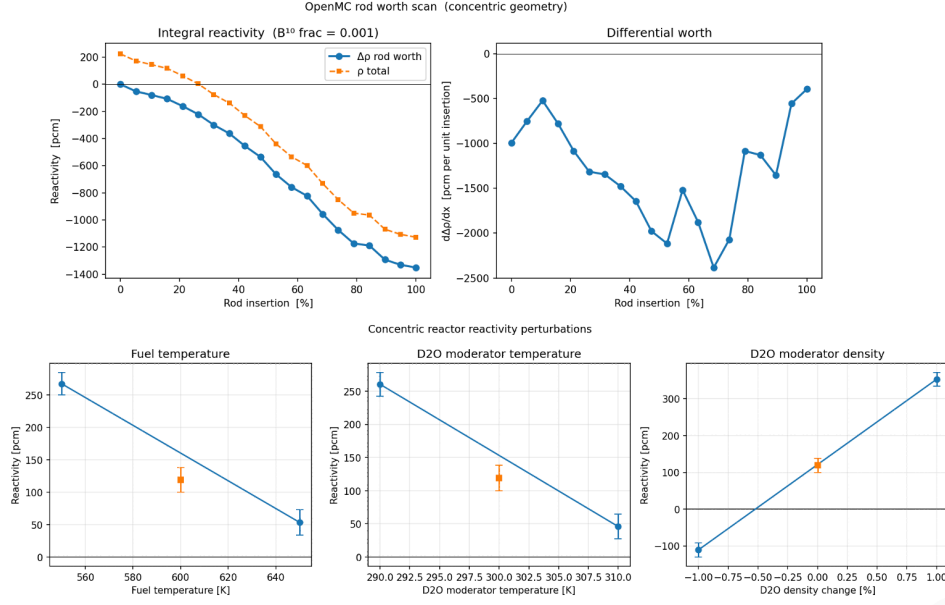


Figure 2: OpenMC perturbations provide the rod-worth and thermal-feedback terms used by point kinetics.

Solving the precursor equation for the new precursor gives

$$C_i^{k+1} = \frac{C_i^k + \Delta t (\beta_i / \Lambda) n^{k+1}}{1 + \lambda_i \Delta t},$$

which is the relation already visible in the source.

Substituting that expression into the neutron balance yields

$$\frac{n^{k+1} - n^k}{\Delta t} = \frac{\rho - \beta}{\Lambda} n^{k+1} + \sum_i \lambda_i \frac{C_i^k + \Delta t (\beta_i / \Lambda) n^{k+1}}{1 + \lambda_i \Delta t}.$$

Now separate the terms involving only known old-state data from the terms that still multiply n^{k+1} :

$$\frac{n^{k+1} - n^k}{\Delta t} = \sum_i \frac{\lambda_i C_i^k}{1 + \lambda_i \Delta t} + \left[\frac{\rho - \beta}{\Lambda} + \sum_i \frac{\lambda_i \Delta t \beta_i / \Lambda}{1 + \lambda_i \Delta t} \right] n^{k+1}.$$

Rearranging gives one scalar solve for the new neutron population,

$$n^{k+1} = \frac{n^k + \Delta t \sum_i \frac{\lambda_i C_i^k}{1 + \lambda_i \Delta t}}{1 - \Delta t \frac{\rho - \beta}{\Lambda} - \Delta t \sum_i \frac{\lambda_i \Delta t \beta_i / \Lambda}{1 + \lambda_i \Delta t}}.$$

In the implementation this matches the code variables `delayed_source`, `delayed_coupling`, `numerator`, and `denominator` in `engine.py`. The precursor update then uses the newly computed n^{k+1} .

Numerically, this matters because the prompt part of the kinetics is very fast relative to the slowest delayed groups. A naive explicit step would require a much smaller Δt for stability, while the semi-implicit update remains well behaved for the chosen browser-friendly step size.

4.7 How the Overview transient is operationalized

The point-kinetics equations are not advanced continuously in a background thread. Instead `service.py` advances them *opportunistically* whenever the frontend polls or sends a command:

1. compute elapsed wall time since the last request,
2. cap that wall-time gap to avoid large jumps after browser pauses,
3. multiply by the configured time-scale factor,
4. then march the engine forward in repeated 0.02 s internal steps.

This architecture is a numerical simplification as much as a software one: the browser sees a continuous transient, but the backend actually performs a sequence of bounded substeps and samples history only every 0.25 s.

The current runtime tuning in `config.py` is:

Parameter	Value
Integrator step	0.02 s
Thermal update batch	0.10 s
Maximum accepted wall-time gap	0.20 s
History sampling	0.25 s
History limit	1200 points
Time scale	8× wall clock
Frontend poll interval	100 ms
Automatic SCRAM threshold	1000 MW _{th}

5 Resolved multigroup diffusion for the Core page

The frontend Core page uses a **resolved multigroup diffusion** chain that derives its nuclear data from the OpenMC continuous-energy calculation. This is the only spatial neutronics visualization exposed in the frontend.

5.1 Overview of the pipeline

The multigroup workflow has five stages, all implemented as reusable Python modules rather than notebook-only logic:

1. **MGXS export** (`openmc/mgxs_export.py`): attach multigroup cross-section tallies to each resolved OpenMC cell (fuel rings, central channel, coolant, moderator, reflector, control rod), run an eigenvalue calculation, and write group constants plus delayed-neutron data as JSON and CSV.
2. **Adapter and region mapping** (`backend/reactor_backend/openmc_mgxs_adapter.py`): parse the OpenMC `model.xml` to extract cell boundaries (z-cylinders and z-planes), load the canonical `mgxs_constants.json`, validate the scattering contract, preserve the centimeter-based OpenMC units, and map each OpenMC cell to a `MultiGroupRegion` inside a `CylindricalLayeredModel2D`.

3. **Finite-volume multigroup diffusion solver** (`backend/reactor_backend/multigroup_diffusion.py`): assemble per-group sparse spatial operators, solve the multigroup eigenvalue problem, and optionally scan rod insertion.
4. **CE-referenced SPH research** (`backend/reactor_backend/multigroup_sph.py`): normalize the OpenMC and diffusion region-integrated fluxes to equal fission production, fit one factor per resolved region and energy group, apply the factors consistently to transport and reaction terms, and evaluate eigenvalue, flux, and power-shape acceptance criteria. This offline workflow is currently paused and its factors are not applied by the Core service.
5. **Persistent preparation cache** (`backend/reactor_backend/multigroup_diffusion_cache.py`): fingerprint the MGXS data, mesh settings, optional offline SPH artifact, and solver configuration; build CSR matrices and a clean-core solution once, then persist them for fast reload.

The command-line entry point is `backend/openmc_to_diffusion.py`. The notebook `openmc/diffusion_concentric_reactor.ipynb` remains the analysis interface, while the web application exposes a four-group rod-aware page backed by the same adapter, clean-state persistent solution, and in-process rodded solves.

5.2 MGXS export from OpenMC

The export module (`mgxs_export.py`) takes a previously built OpenMC `model.xml` and attaches multigroup tally objects to every resolved cell domain. The domain definitions are configurable; the resolved mode maps each `fuel_ring_N` cell to its own domain label, plus the central moderator channel, heavy-water coolant, outer moderator, reflector, and parked control rod. A frozen `MGXSExportConfig` dataclass captures the particle count, batch schedule, energy-group edges (any CASMO structure from 1 to 70 groups), Legendre scattering order (P0, P1, P3, or P5), and the delayed-neutron group count.

Two scattering representations are tallied from the same continuous-energy run:

- The **canonical diffusion export** uses non- ν , uncorrected scattering (`scatter_correction = null`). The adapter prefers the **consistent scatter matrix** and falls back to **scatter matrix**. Only the zeroth Legendre moment is passed to diffusion; higher moments are recorded and discarded.
- A **supplemental P0-corrected consistent scattering library** is tallied for OpenMC’s own multigroup transport validation run. This correction is validation-only and is not written into the canonical diffusion JSON.

When `validation_scatter_correction = P0`, the export contract requires `legendre_order = 0`, because OpenMC does not apply that correction to a higher-order Legendre library. A higher-order canonical export can still be read by the adapter, but the validation correction must be disabled for that run and the diffusion path will retain only moment zero.

The diffusion coefficient is taken directly from OpenMC’s transport cross-section tally:

$$D_g = \frac{1}{3 \Sigma_{tr,g}}.$$

This distinction is important. OpenMC’s P0 scatter correction modifies the isotropic scattering data used by its multigroup *transport* calculation to represent part of the first angular moment. The

diffusion calculation instead represents anisotropic migration through $\Sigma_{tr,g}$ in D_g . Its scattering source therefore uses the uncorrected non- ν P0 reaction-rate matrix. The two calculations use different angular closures; agreement of the OpenMC multigroup validation does not imply that the diffusion approximation has been calibrated.

All neutronics quantities remain in OpenMC’s native centimeter-based units: D in cm, macroscopic cross sections in cm^{-1} , and geometry and mesh dimensions in cm. No SI conversion is performed in the adapter. This gives $D\nabla^2\phi$ and $\Sigma\phi$ consistent units throughout the steady solver.

Each export run produces:

- `mgxs_constants.json` — nested group constants, delayed-neutron data, metadata for every domain, raw region/group CE flux tally means and standard deviations, and a cylindrical CE kappa-fission power-mesh tally;
- `group_constants.csv` — flat table of all group-constant rows;
- `delayed_neutrons.csv` — flat table of all delayed-neutron rows.

The raw CE flux tally is retained in fast-to-thermal group order without the independent group-wise normalization used by the GeN-Foam auxiliary export. This distinction is necessary because SPH requires relative region/group reaction rates from one common Monte Carlo normalization. The cylindrical power tally supplies independent normalized radial and axial shape checks.

The notebook `openmc/exportMGXS.ipynb` runs group-count and Legendre-order sweeps, storing each result under `group_sweep/group_<N>/`. It also runs an OpenMC multigroup validation step that rebuilds an `mgxs.h5` library and solves the same geometry in multigroup transport mode, producing the $\Delta\rho$ (MG – CE) values reported in the workflow document.

5.3 Adapter: from OpenMC cells to diffusion regions

The adapter (`openmc_mgxs_adapter.py`) is the bridge between the OpenMC export and the diffusion solver. It performs five operations:

1. **Geometry extraction.** Parses the OpenMC `model.xml` with Python’s `ElementTree`, identifies every named cell, and extracts its radial and axial bounds from the z-cylinder and z-plane surfaces referenced in the cell’s region specification.
2. **Contract validation.** Requires `scatter_correction = null` in the export config so the P0 scattering diagonal is not transport-corrected. Rejects exports that are missing required domains (fuel rings, central channel, moderator, reflector) or whose MGXS domain labels do not match the XML cell names.
3. **Scattering-matrix selection.** Prefers consistent `scatter matrix`; falls back to `scatter matrix`. Discards any Legendre moments above P0 and interprets the stored matrix as $\Sigma_{s,g'\rightarrow g}$, with the first group index denoting the source group and the second the destination group.
4. **Diffusion-coefficient validation.** For every active region/group pair, checks that the exported coefficient is finite, positive, and consistent with $D_g = 1/(3\Sigma_{tr,g})$ to numerical precision.
5. **Inactive-group sanitization.** Energy groups with zero absorption, zero fission, zero chi, and zero scatter coupling are flagged as inactive. Their undefined diffusion coefficients are

replaced with a finite coefficient from another active group so the spatial operator remains well-posed; every replacement is reported in the adapter summary.

The output is a `CylindricalLayeredModel2D` containing one `MultiGroupRegion` per OpenMC cell. Each region holds per-group vectors for D , Σ_a , $\nu\Sigma_f$, $\kappa\Sigma_f$, χ , and a $G \times G$ P0 scattering matrix $\Sigma_{s,g' \rightarrow g}$. The adapter also loads the optional raw CE region/group flux and power-mesh reference. The radial mesh is fitted exactly to all XML-derived material boundaries so no interface is lost to grid coarsening. “Resolved” here means that no new cross-cell homogenization is introduced: each exported cell receives its own constants. The constants are still flux-weighted averages within that OpenMC tally cell, so this is not a pin-resolved or continuously varying material description.

5.4 Finite-volume multigroup diffusion solver

The solver in `multigroup_diffusion.py` solves the G -group steady-state diffusion eigenvalue problem on a 2D cylindrical (r, z) mesh:

$$-\nabla \cdot (D_g \nabla \phi_g) + \Sigma_{a,g} \phi_g + \sum_{g' \neq g} \Sigma_{s,g \rightarrow g'} \phi_{g'} = \frac{\chi_g}{k_{\text{eff}}} \sum_{g'} \nu \Sigma_{f,g'} \phi_{g'} + \sum_{g' \neq g} \Sigma_{s,g' \rightarrow g} \phi_{g'}.$$

The left-hand side collects leakage, absorption, and outscatter from group g into a single loss operator A_g . The right-hand side contains the fission source (scaled by $1/k_{\text{eff}}$) and in-scatter from all other groups. Within-group scattering does not appear explicitly because its loss and gain terms cancel in the group balance. Thus the removal coefficient used in A_g is

$$\Sigma_{r,g} = \Sigma_{a,g} + \sum_{g' \neq g} \Sigma_{s,g \rightarrow g'}.$$

5.4.1 Spatial discretization

The mesh is a nonuniform, cell-centered finite-volume grid in cylindrical coordinates. Every material radius and axial plane extracted from `model.xml` is inserted as a mesh face before the intervals are subdivided. The target radial spacing is region dependent (fuel, core coolant, moderator, and reflector), while one target axial spacing is currently used throughout.

Integrating the diffusion equation over cell (i, j) gives a balance between currents through four faces and volumetric reaction terms. Omitting the common factor 2π , the cell volume is

$$V'_{ij} = \frac{1}{2} (r_{i+1/2}^2 - r_{i-1/2}^2) \Delta z_j.$$

For two adjacent cells P and N , the face conductance is

$$H_{PN} = \left(\frac{\delta_P}{D_P} + \frac{\delta_N}{D_N} \right)^{-1},$$

where δ_P and δ_N are the center-to-face distances. This is the harmonic interface treatment implied by continuity of normal current. After division by V'_{ij} , the off-diagonal coefficient is the negative face conductance multiplied by the cylindrical face area. For example,

$$C_{i+1/2,j} = \frac{r_{i+1/2} H_{i,i+1}}{\frac{1}{2}(r_{i+1/2}^2 - r_{i-1/2}^2)},$$

and the corresponding axial coefficient is $C_{i,j+1/2} = H_{j,j+1}/\Delta z_j$. The diagonal contains the sum of all outgoing leakage coefficients plus $\Sigma_{r,g}$.

The assembled per-group loss operator A_g is a sparse CSR matrix with a five-point stencil in (r, z) . It is factorized once per solve with a sparse direct solver (SuperLU via `scipy.sparse.linalg.factorized`).

5.4.2 Production-mesh qualification

The online Core page intentionally matches `openmc/diffusion_concentric_reactor.ipynb`. Its target spacing is

$$(\Delta r_{\text{fuel}}, \Delta r_{\text{cool}}, \Delta r_{\text{mod}}, \Delta r_{\text{refl}}, \Delta z) = (0.1, 1, 5, 20, 10) \text{ cm},$$

which preserves every material face and produces 13 500 cells.

The separate offline SPH study used a finer 10 cm reflector target. Because SPH factors are mesh dependent, that study selected its mesh before fitting. It compared the 15 096-cell reference

$$(0.1, 1, 5, 10, 10) \text{ cm}$$

with a 6 360-cell candidate using $(0.5, 1, 5, 10, 20)$ cm targets. The candidate changed k_{eff} by only +5.52 pcm but failed the field criteria: the maximum resolved region/group flux error was 35.3% in the parked rod and the radial and axial power-shape RMS errors were 4.30% and 3.07%. The 15 096-cell mesh therefore remains the archived SPH fitting reference. It is not the mesh used by the live Core service, and no SPH factors are loaded online.

Power profiles are compared as power per unit radial or axial coordinate rather than raw power per mesh bin. Otherwise changing the bin width would appear as a physical shape change.

5.4.3 Boundary conditions

At $r = 0$, the missing west-face term imposes the cylindrical symmetry condition

$$\left. \frac{\partial \phi_g}{\partial r} \right|_{r=0} = 0.$$

The Core service uses the `groupwise_fvm` vacuum treatment. The mesh ends at the physical H₂O reflector boundary and each energy group receives a finite-volume boundary-leakage term corresponding to the extrapolation length

$$d_{\text{ext},g} = 2.13 D_{\text{refl},g}.$$

This retains each group's reflector diffusion coefficient instead of extending all groups to one common artificial boundary.

The solver also retains `extrapolated_mesh` for comparison, where the mesh is extended by $2.13 \max_g D_{\text{refl},g}$ and zero flux is imposed at that boundary. A `d2o_interface_vacuum` mode truncates at the D₂O/H₂O interface for diagnostic sensitivity studies. Neither alternate mode is the default webapp calculation.

5.4.4 Eigenvalue iteration

The multigroup eigenvalue problem is solved by nested iteration:

1. **Outer power iteration.** Start from a positive flux guess and an initial $k_{\text{eff}} = 1$. At each outer iteration:

- (a) compute the fission density $f = \sum_g \nu \Sigma_{f,g} \phi_g$;

- (b) form the fixed fission source $s_g^{\text{fiss}} = \chi_g f / k_{\text{eff}}$;
 - (c) solve the coupled scattering system $A_g \phi_g = s_g^{\text{fiss}} + \sum_{g' \neq g} \Sigma_{s,g' \rightarrow g} \phi_{g'}$ for all groups.
2. **Inner scattering iteration.** The coupled system first receives at most five Gauss–Seidel sweeps over destination groups. The requested inner tolerance is scheduled from 10^{-2} in early outer iterations to the user-specified `inner_tol`.
 3. **Matrix-free GMRES fallback.** If those Gauss–Seidel sweeps do not satisfy the active inner tolerance, the code solves the coupled scattering system with matrix-free GMRES, preconditioned by the same per-group SuperLU factors. This avoids assembling a monolithic $(G \times N_{\text{cells}}) \times (G \times N_{\text{cells}})$ matrix, which would produce prohibitive fill-in for higher group counts and fine meshes.
 4. **Eigenvalue update.** After each outer iteration the eigenvalue is updated from the ratio of new to old total fission production,

$$k_{\text{new}} = k_{\text{old}} \frac{\sum_{i,j} V_{ij} \sum_g \nu \Sigma_{f,g,ij} \phi_{g,ij}^{\text{new}}}{\sum_{i,j} V_{ij} \sum_g \nu \Sigma_{f,g,ij} \phi_{g,ij}^{\text{old}}}.$$

5. **Flux normalization.** The flux is normalized so the volume-integrated fission production rate equals unity,

$$\sum_{i,j} V_{ij} \sum_g \nu \Sigma_{f,g,ij} \phi_{g,ij} = 1.$$

This fixes the otherwise arbitrary eigenvector scale and improves numerical conditioning. It is not an absolute power normalization; conversion to physical flux or power requires a separate amplitude based on reactor power and recoverable energy per fission.

Convergence is declared only after the final requested inner tolerance is active and both

$$\frac{|k_{\text{new}} - k_{\text{old}}|}{|k_{\text{new}}|} < \text{tol}$$

and the volume-weighted change in fission-source shape is below `source_tol`. A volume-weighted equation-balance residual is computed after convergence and reported as a diagnostic, but it is not currently a stopping criterion.

5.5 CE-referenced superhomogenization

The SPH implementation uses one positive factor $\mu_{i,g}$ for each resolved OpenMC region i and energy group g . Raw CE and diffusion region-integrated scalar fluxes are first normalized so that both have unit total fission production:

$$\sum_{i,g} \nu \Sigma_{f,i,g} \Phi_{i,g} = 1.$$

For the current iterate, the direct fixed-point target is

$$\mu_{i,g}^* = \frac{\Phi_{i,g}^{\text{CE}}}{\Phi_{i,g}^{\text{diff}}}.$$

The code damps this target in logarithmic space,

$$\ln \mu_{i,g}^{(n+1)} = (1 - \alpha) \ln \mu_{i,g}^{(n)} + \alpha \ln \mu_{i,g}^*, \quad \alpha = 0.5,$$

which preserves positivity and avoids an asymmetric additive update.

For each source group, the factor multiplies absorption, fission, **kappa-fission**, and the complete scattering row:

$$\begin{aligned} \tilde{\Sigma}_{a,i,g} &= \mu_{i,g} \Sigma_{a,i,g}, & \tilde{\nu \Sigma}_{f,i,g} &= \mu_{i,g} \nu \Sigma_{f,i,g}, \\ \tilde{\Sigma}_{s,i,g \rightarrow g'} &= \mu_{i,g} \Sigma_{s,i,g \rightarrow g'}. \end{aligned}$$

The transport term is transformed by the same factor. Since the solver stores $D = 1/(3\Sigma_{\text{tr}})$, this becomes

$$\tilde{D}_{i,g} = \frac{D_{i,g}}{\mu_{i,g}}.$$

The fission spectrum χ is unchanged. Thus the fitting objective is flux and reaction-rate equivalence, not direct adjustment of k_{eff} .

Only statistically active bins enter convergence and qualification. The fit requires active transformed flux error and factor-log changes below 0.2% for two successive iterations; bounded factors and an iteration limit make failure explicit. The immutable **SphFactorSet** records the factor matrix, active mask, convergence history, mesh, source fingerprint, algorithm version, and provisional state. A factor artifact is rejected when the MGXS JSON, XML geometry, mesh, or algorithm version changes.

The current default qualification thresholds are 1% maximum active region/group flux error, 8% radial power-shape RMS error, and 10% axial power-shape RMS error. Corrected k_{eff} and the CE uncertainty are reported for diagnosis but are not qualification gates. These criteria apply to the offline SPH workflow only; the online Core service does not load an SPH factor set.

5.5.1 Post-solve power-shape correction

The current production correction is not SPH. It is a post-solve power-map correction used only after the diffusion flux and eigenvalue have been computed. The clean continuous-energy kappa-fission mesh and the clean diffusion power map are normalized by total power before forming

$$C_{\text{clean}}(r, z) = \frac{\hat{P}_{\text{CE}}(r, z; 0)}{\hat{P}_{\text{diff}}(r, z; 0)}.$$

For a later rod position x , the corrected map keeps this factor fixed:

$$\hat{P}_{\text{map}}(r, z; x) = \text{normalize} [C_{\text{clean}}(r, z) P_{\text{diff}}(r, z; x)].$$

This is equivalent to applying the relative rod-induced shape change predicted by diffusion to the clean CE power shape. The heat-source map used by a point-kinetics coupling is then

$$q'''(r, z, t) = P_{\text{PK}}(t) \hat{P}_{\text{map}}(r, z; x(t)).$$

The correction does not change MGXS, flux, or k_{eff} .

5.5.2 Rod-worth scan

The equivalent-absorber rod model adds a scalar $\Delta\Sigma_a$ inside the physical rod radius above an axially moving rod tip. In those cells it also sets $\nu\Sigma_f$, χ , and scattering to zero. This is an equivalent black/absorbing-region perturbation, not a material replacement using the parked-rod MGXS. A rod scan sweeps the insertion fraction $x \in [0, 1]$, rebuilding and factorizing the affected group operators at each position. The previous converged flux may be used as the next initial guess.

The resulting $\Delta\rho(x)$ is explicitly exploratory. Backend point kinetics uses the OpenMC CE rod-worth CSV instead. The equivalent absorber is kept because it gives a practical rodged diffusion power shape, but it is not a material replacement using rodged MGXS. The present limitations are that the available MGXS were generated for the parked rod geometry, the physical rod is shorter than the desired full-height absorber representation, and the absorber composition used for the OpenMC worth scan was intentionally weakened relative to default B4C. Clean-core SPH factors, when available, do not qualify this rod perturbation.

5.6 Persistent preparation cache

The full solve chain (adapter $\rightarrow$ mesh construction $\rightarrow$ per-group CSR assembly $\rightarrow$ convergence) is expensive, especially for higher group counts and fine meshes. The cache module (`multigroup_diffusion_cache.py`) avoids recomputing this work on every notebook restart or explicit cache preparation.

The cache fingerprint is a SHA-256 hash of:

- the canonical `mgxs_constants.json` contents,
- the colocated `model.xml` contents,
- the mesh-spacing configuration (target Δr and Δz per region),
- the complete SPH factor artifact, including its algorithm version, when corrections are applied,
- the rod-absorption parameter and solver tolerances,
- and an explicit cache-schema version.

The fingerprint does not hash the Python source itself. A numerical algorithm change that affects cached data therefore requires incrementing the cache schema version.

On a cache hit, the following are loaded directly from disk:

- radial and axial mesh edge arrays,
- cell-wise material arrays (D , Σ_a , $\nu\Sigma_f$, $\kappa\Sigma_f$, χ , P0 scattering, and resolved-region indices),
- one CSR spatial matrix per energy group,
- and the converged clean-core eigenvalue, flux field, and power density.

SuperLU factorization objects are process-local and are not serialized; a process that needs to solve again (e.g. for a rod scan) loads the CSR matrices and re-factorizes them in memory. The persisted operators describe the clean state only. A changed rod position alters the absorption and scattering maps, so its operators must still be rebuilt and factorized. The web service keeps those rodged multigroup solves in an in-process cache keyed by insertion fraction, but they are not

written to the persistent preparation cache. On a cache miss, the full clean-state build-and-solve pipeline runs synchronously and the results are persisted.

The default cache root is `backend/.cache/openmc_diffusion/`. The CLI entry point `backend/openmc_to_diffusion.py` exposes a `-qualify-mesh` study, `-fit-sph` factor generation, and `-prepare-cache` operator preparation for use outside the notebook.

5.7 Integration with the web application

The web backend now exposes the four-group rod-aware model through:

- GET `/api/multigroup-diffusion/state`, and
- POST `/api/multigroup-diffusion/recompute`.

The service is initialized lazily. Its default path intentionally matches the validated notebook in `openmc/diffusion_concentric_reactor.ipynb`: four energy groups, the `groupwise_fvm` boundary treatment, 0.1/1/5/20 cm radial mesh targets for fuel/coolant/moderator/reflector, and a 10 cm axial target. The clean unrodded solve gives $k_{\text{eff}} = 1.001429$ (+142.7 pcm relative to critical), within about 1 pcm of the clean continuous-energy OpenMC reference.

The flux map shown by the frontend is the raw four-group diffusion scalar flux, mirrored from cylindrical r - z into an axisymmetric x - z image. Power is displayed with the fixed clean CE power-shape factor when a CE power mesh is available. Each response reads the current rod insertion from the point-kinetics simulation, solves or reuses the corresponding rodded multigroup diffusion state, and applies the fixed clean factor to the rodded diffusion power map. The frontend names this view **Core**; the backend route names remain `multigroup-diffusion` for compatibility.

SPH fitting and qualification remain available in the offline workflow, but the online Core page does not apply SPH factors. Rodded maps are marked provisional because they use an equivalent absorber, not rodded MGXS or rod-dependent transport corrections. Reactivity remains tied to the OpenMC CE rod-worth CSV, not to the equivalent diffusion rod-worth scan.

5.8 Modelica thermal-hydraulics coupling

`SimulationService` couples point kinetics to the Modelica model through `ThermalAdapter`. Every thermal batch sends total point-kinetics power and eight rod-aware axial power fractions to the FMU. Those fractions currently come from `core_service.compute_axial_power_fractions_8()`, which evaluates the retired one-group diffusion shape. The four-group CE-corrected Core map does not yet drive the FMU heat-source distribution. The adapter reads core inlet and outlet temperature, effective fuel and moderator temperature, effective moderator density, maximum fuel temperature, mass flow, and core pressure drop. The effective fuel temperature, moderator temperature, and density are returned to the point-kinetics reactivity calculation.

The FMU is advanced at 0.1 s simulated-time intervals using the average power over each batch. Thermal feedback is applied only when the returned snapshot has source `fm_u`; the fallback thermal model can keep the HMI populated but contributes zero feedback reactivity.

The thermal model is an aggregated system model rather than the OpenMC geometry itself. Its current fuel heat structure uses six equivalent rings, 0.5 cm fuel thickness, 7 cm coolant gaps, radii 4.5–50 cm, and a 1.5 m active height. The published neutronics reference uses seven resolved fuel rings and a 3.0 m active height. This geometry mismatch is a current coupling limitation: power and effective feedback signals are exchanged, but material regions are not mapped one-to-one, and the axial heat-source input still comes from the legacy spatial helper. The thermal

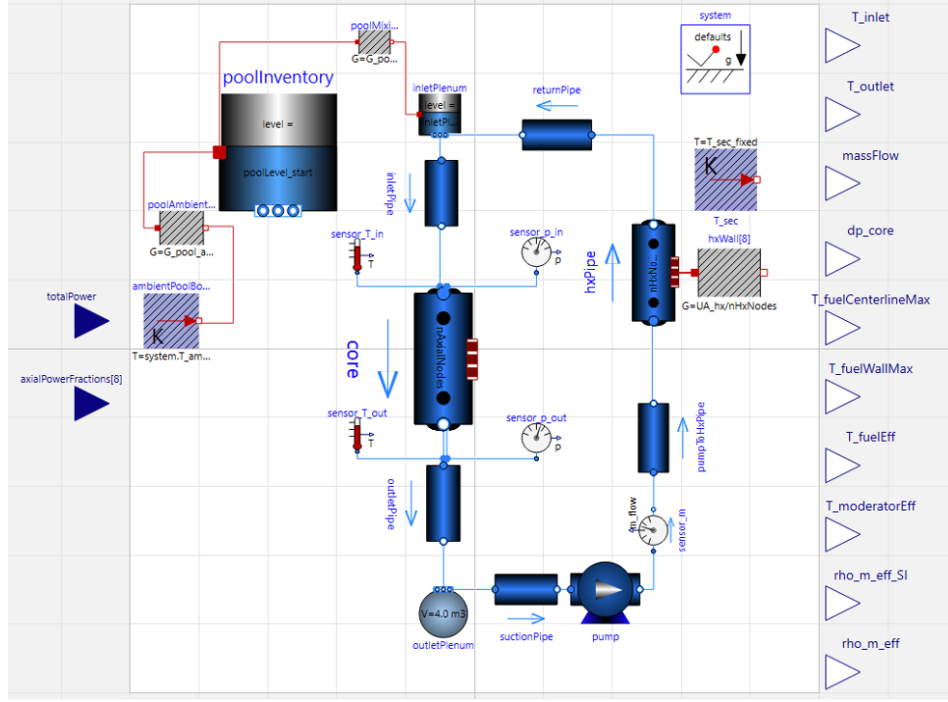


Figure 3: Modelica thermal-hydraulics model coupled to the Python backend through an FMI 2.0 co-simulation FMU.

equations and FMU interface are documented in `docs/TH_latex/ThermalHydraulics.pdf` and `docs/fmu_coupling.md`.

5.9 Current status and verification

The adapter and export configuration support the listed 1, 2, 4, 8, 16, 25, 40, and 70 group structures. The documented diffusion benchmark table currently contains resolved clean-core results through 16 groups; support for a group structure should not be confused with a completed mesh/runtime verification at that group count. The workflow document `docs/diffusion_mgx_workflow.md` records the current comparison:

- OpenMC multigroup transport with the validation-only P0 correction approaches the continuous-energy reference as the group structure is refined: the reported MG biases are +806, +426, +166, and +66 pcm for 2, 4, 8, and 16 groups, respectively.
- The uncorrected diffusion bias (diffusion k_{eff} minus OpenMC MG k_{eff}) stabilizes around -400 to -500 pcm for the documented 4, 8, and 16 group cases. This is evidence of a repeatable residual bias, not proof that diffusion closure alone is its cause.
- The resolved two-group diffusion result is anomalous relative to that trend: its bias is about +1194 pcm relative to two-group OpenMC MG. This points to sensitivity to the very broad energy condensation and groupwise diffusion coefficients rather than a simple monotonic group-count convergence law.
- The resolved one-group diffusion result carries an approximately +1800 pcm bias relative to continuous energy. Because the fuel rings remain spatially resolved in this calculation, that

result cannot be attributed to collapsing the rings into one spatial region.

The clean online result is qualified by the current service because its $k_{\text{eff}} = 1.001429$ is within about 1 pcm of the associated continuous-energy reference, well inside the 21.75 pcm one-sigma uncertainty. The clean displayed power shape uses the CE correction described above. This qualification is specific to the clean four-group webapp calculation; it is not an SPH qualification.

The SPH fitting, application, fingerprinting, cache, and qualification machinery are implemented and unit tested, but the workflow is paused. Trial factor sets are archived for later investigation and are not loaded online. Rodded Core results remain provisional, and discontinuity factors, true rodded MGXS, and rod-dependent transport corrections remain deferred.

6 How the modeling layers relate

Layer	Model	Purpose
OpenMC MGXS export	Resolved-cell multigroup	Generate transport-derived group constants and scattering matrices from continuous-energy Monte Carlo
Reference-data loaders	JSON, XML, and CSV adapters	Validate and provide immutable OpenMC-derived runtime inputs
Overview page	0D six-group point kinetics	Evolve reactor amplitude from rod worth and FMU thermal feedback
Modelica FMU	Reduced primary-loop and fuel heat structure	Convert power and axial shape into coolant, fuel, and moderator feedback state
Core page	Four-group 2D finite-volume diffusion	Solve and display resolved-cell flux and CE-corrected power shape

The runtime workflow is:

1. publish OpenMC reference artifacts offline;
2. initialize point kinetics at the CE critical rod position;
3. advance point kinetics and batch its power into the Modelica FMU;
4. feed effective FMU temperatures and density back into reactivity;
5. solve or reuse the four-group diffusion field at the current rod position for the Core page;
6. apply the fixed clean CE power-shape correction to the Core display;
7. separately derive the FMU axial fractions from the legacy one-group shape helper.

7 Important assumptions and limitations

The current application remains an educational simulator:

- point kinetics assumes one global neutron-population amplitude and six lumped precursor groups;

- the displayed absolute flux scale is inherited from the legacy one-group model;
- one effective rod bank stands in for a detailed control system;
- rodded diffusion uses equivalent absorption and a fixed clean CE power correction rather than rodded MGXS;
- the Modelica heat structure is aggregated and does not share a one-to-one mesh or geometry with the OpenMC model;
- the FMU axial power fractions still come from the legacy one-group diffusion helper rather than the four-group CE-corrected map;
- fallback thermal values do not contribute reactivity feedback;
- no xenon, burnup, fuel depletion, boiling/void feedback, or detailed secondary-side control is modeled;
- discontinuity factors and runtime SPH are not applied;
- time-dependent multigroup solves are not implemented in the web backend.

The resolved MGXS remain flux-weighted averages within each OpenMC tally cell, even though the adapter preserves every exported cell as a separate diffusion region. The model is not pin-resolved and is not a licensing-grade core code.

8 Conclusion

OpenMC defines the current reactor and publishes the compact data consumed by the application. Point kinetics provides fast amplitude dynamics, the Modelica FMU closes the temperature and density feedback loop, and the four-group finite-volume solver provides the spatial Core view. The clean diffusion result reproduces its associated CE eigenvalue and uses CE data to correct the displayed power shape. The rodded spatial treatment and absolute flux normalization remain the clearest next physics improvements.

The executable interfaces to this model are `backend/openmc_to_diffusion.py`, `openmc/diffusion_concentric_reactor.ipynb`, and the two webapp pages. Detailed operational contracts are kept in `docs/diffusion_mgxs_workflow.md`, `docs/fmu_coupling.md`, and `docs/system_architecture.md`.